

Real-Time Video Enhancement Pipeline

Frame Interpolation and Super-Resolution without Motion Vectors

*A DLSS-Inspired Deep Learning Approach
for Low-Resolution and Low-FPS Videos*



Comprehensive Project Report

Submitted by

Gayatri Priya – EP23BTECH11032

Krishna Teja – CS23BTECH11028

Jai Prakash – CS23BTECH11015

Machine Learning for Physical Sciences

Indian Institute of Technology Hyderabad

[GitHub Repository Link](#)

Submitted to

Sangkha Borah

April 2026

Abstract

Modern video enhancement systems aim to improve both spatial resolution and temporal resolution. Spatial resolution determines the sharpness and clarity of individual frames, while temporal resolution determines the smoothness of motion across frames. Commercial systems such as NVIDIA DLSS achieve high-quality enhancement by using additional rendering information such as motion vectors, depth buffers, and temporal history. However, normal video files do not contain such auxiliary information. This project therefore explores whether a DLSS-like video enhancement system can be constructed using only the available video frames.

The proposed system takes a low-resolution, low-frame-rate video as input and produces a high-resolution, high-frame-rate video as output. The pipeline consists of two main deep learning components: a frame interpolation model and a super-resolution model. The frame interpolation model predicts missing intermediate frames between consecutive input frames, thereby increasing the effective frame rate. The super-resolution model increases the spatial resolution of frames using an Efficient Sub-Pixel Convolutional Neural Network (ESPCN). Together, these two models improve both motion smoothness and image clarity.

The frame interpolation model learns motion directly from pairs of input frames. Since explicit motion vectors are unavailable, the model estimates motion implicitly using convolutional feature extraction, flow-like displacement prediction, mask-based blending, differentiable warping, and residual refinement. The super-resolution model uses ESPCN, which performs most computations in low-resolution space and applies PixelShuffle at the final stage to generate a $4\times$ upscaled output. This design enables high-speed inference suitable for real-time or near-real-time video processing.

The complete pipeline is implemented using ONNX Runtime. Both trained models are exported to ONNX format and loaded into a unified video processing system. The pipeline includes hardware detection, ONNX provider selection, frame reading, normalization, padding, tensor conversion, interpolation, upscaling, output reconstruction, progress reporting, performance estimation, preview support, caching logic, codec handling, and optional audio preservation using FFmpeg.

The upscaling model achieved an average PSNR of 31.02 dB, SSIM of 0.9847, MSE of 0.001286, MAE of 0.018320, RMSE of 0.032120, inference time of 0.97 ms per image, and estimated throughput of 1031 FPS on an NVIDIA RTX 3050 Laptop GPU. The frame interpolation model achieved PSNR of 33.8689 dB, SSIM of 0.9521, MS-SSIM of

0.9832, LPIPS of 0.0495, MSE of 0.000769, RMSE of 0.0240, average latency of 14.68 ms, maximum throughput of 68.1 FPS, PSNR standard deviation of 5.2640, and peak VRAM usage of 260.30 MB.

This report presents the theoretical foundation, model design, dataset preparation, training methodology, deployment strategy, pipeline architecture, evaluation metrics, experimental results, visual analysis, limitations, and future scope of the complete system.

This work combines two major components: frame interpolation for generating intermediate frames and image super-resolution for improving the spatial quality of individual frames. These components are integrated into a unified video enhancement pipeline that takes an input video, processes its frames, enhances spatial clarity and temporal smoothness, and generates an improved output video. The project demonstrates a practical approach to DLSS-inspired video enhancement using only ordinary video frames, without requiring motion vectors, depth buffers, or other auxiliary rendering information.

Contributions

The project was carried out as a collaborative effort, with each team member contributing to a specific component of the proposed video enhancement system. The individual contributions are summarized below:

Individual Contributions

Name	Roll Number	Contribution
Krishna Teja	CS23BTECH11028	Responsible for the frame generation and frame interpolation component of the project. This involved generating intermediate frames between consecutive video frames to improve temporal resolution and motion smoothness.
Gayatri Priya	EP23BTECH11032	Responsible for the image upscaling and super-resolution component of the project. This included implementing and evaluating the ESPCN-based enhancement model and analyzing the improvement in spatial resolution using quantitative and visual metrics.
Jai Prakash	CS23BTECH11015	Responsible for pipeline integration and video-level testing. This included combining the frame interpolation and image upscaling modules into a complete processing pipeline and evaluating the final enhanced video output.

Contents

Abstract	i
Contributions	iii
1 Introduction	1
1.1 Motivation	1
1.2 Inspiration from DLSS	1
1.3 Problem Statement	2
1.4 Core Pipeline Idea	2
1.5 Project Objectives	3
1.6 Report Organization	3
2 Fundamentals of Digital Video Processing	4
2.1 Video as a Spatio-Temporal Signal	4
2.2 Frame Representation	4
2.3 Spatial Resolution	5
2.4 Temporal Resolution	5
2.5 Why Video Enhancement is Ill-Posed	5
2.6 Spatial and Temporal Reconstruction	5
3 Mathematical Formulation of the Enhancement Problem	7
3.1 Spatial Degradation Model	7
3.2 Temporal Degradation Model	7
3.3 Joint Enhancement Model	8
3.4 Ordering of Operations	8
3.5 Error Propagation	8
4 Theory of Frame Interpolation	9
4.1 Definition	9

4.2	Simple Averaging and Its Failure	9
4.3	Optical Flow	9
4.4	Aperture Problem	10
4.5	Occlusion	10
4.6	Warping	10
4.7	Mask-Based Blending	10
4.8	Residual Refinement	11
5	Theory of Super-Resolution	12
5.1	Definition	12
5.2	Ill-Posed Nature	12
5.3	Frequency Domain View	12
5.4	Traditional Upscaling	12
5.5	Deep Learning Based Super-Resolution	13
6	Frame Interpolation Model	14
6.1	Input and Output	14
6.2	Encoder	14
6.3	Residual Blocks	14
6.4	Decoder	15
6.5	Flow and Mask Separation	15
6.6	Base Frame Prediction	15
6.7	Refinement Network	15
6.8	Why This Architecture Works	16
7	Super-Resolution Model: ESPCN	17
7.1	Why ESPCN	17
7.2	Architecture	17
7.3	PixelShuffle	18
7.4	Advantages	18
8	Dataset and Preprocessing	19
8.1	Frame Interpolation Dataset	19
8.2	Super-Resolution Dataset	19
8.3	Random Cropping	19
8.4	Flipping	20

8.5	Normalization	20
9	Training Methodology	21
9.1	Frame Interpolation Training	21
9.2	Charbonnier Loss	21
9.3	Laplacian Pyramid Loss	21
9.4	Combined Loss	21
9.5	Super-Resolution Training	22
9.6	Optimizers	22
10	ONNX Export and Deployment	23
10.1	Need for ONNX	23
10.2	Dynamic Axes	23
10.3	ONNX Runtime Providers	23
10.4	CPU Fallback	23
11	Complete Pipeline Architecture	24
11.1	Pipeline Overview	24
11.2	Hardware Detection	24
11.3	Model Loading	24
11.4	Video Reading	25
11.5	Frame Pair Processing	25
11.6	Normalization and Tensor Conversion	25
11.7	Padding	25
11.8	Interpolation Stage	25
11.9	Upscaling Stage	25
11.10	Video Reconstruction	26
11.11	Audio Preservation	26
12	Performance Estimation and Optimization	27
12.1	Why Performance Estimation is Needed	27
12.2	Latency	27
12.3	Throughput	27
12.4	Preview Mode	27
12.5	Caching	28
12.6	Batch Size Detection	28

13 Evaluation Metrics	29
13.1 Need for Multiple Metrics	29
13.2 MSE	29
13.3 MAE	29
13.4 RMSE	29
13.5 PSNR	29
13.6 SSIM	29
13.7 MS-SSIM	30
13.8 LPIPS	30
13.9 Latency and FPS	30
13.10 Model Size and Parameter Count	30
14 Experimental Results	31
14.1 Upscaling Model Results	31
14.2 Interpretation of Upscaling Results	31
14.3 Frame Interpolation Results	31
14.4 Interpretation of Interpolation Results	32
14.5 Combined Pipeline Interpretation	32
15 Visual Analysis	33
15.1 Importance of Visual Results	33
15.2 Frame Interpolation Comparison	33
15.3 Super-Resolution Comparison	34
15.4 Zoomed Crop Analysis	34
15.5 Metric Graphs	34
15.5.1 Super-Resolution Metric Analysis	35
15.5.2 Frame Interpolation Metric Analysis	37
15.5.3 Combined Metric Summary	40
15.5.4 Overall Interpretation	41
16 Development Timeline	42
16.1 Initial Stage	42
16.2 Dataset Setup	42
16.3 Model Exploration	42
16.4 Loss Improvements	42
16.5 ONNX Deployment	42

16.6 Pipeline Integration	42
16.7 Final Improvements	43
17 Limitations	44
17.1 Frame Interpolation Limitations	44
17.2 Super-Resolution Limitations	44
17.3 Pipeline Limitations	44
18 Future Work	45
19 Conclusion	46

Chapter 1

Introduction

1.1 Motivation

Video is one of the most widely used forms of digital information. It is used in entertainment, education, communication, surveillance, sports analysis, scientific visualization, online meetings, and real-time streaming. The perceived quality of a video depends mainly on two factors: spatial quality and temporal quality. Spatial quality refers to how detailed and sharp each frame appears. Temporal quality refers to how smoothly motion is represented across time.

A low-resolution video lacks fine spatial details. Edges appear blurred, textures are lost, small objects become unclear, and the visual experience becomes poor. A low-frame-rate video, on the other hand, suffers from discontinuous motion. Moving objects appear jumpy because there are too few frames representing their motion.

In many real-world cases, both problems appear together. A video may be low-resolution and low-frame-rate because of old recording devices, bandwidth limitations, video compression, low-light capture, camera limitations, or storage constraints. Therefore, a useful enhancement system should improve both the resolution of frames and the smoothness of motion.

This project aims to build such a system using deep learning.

1.2 Inspiration from DLSS

NVIDIA DLSS is a well-known deep learning based technology used in real-time graphics. It reconstructs high-quality frames by using low-resolution rendered frames along with auxiliary information such as motion vectors, depth buffers, and previous frame history. Motion vectors explicitly tell the system how objects and pixels move between frames.

However, our problem is different. We are not working inside a game engine or rendering pipeline. We are working with normal videos. Such videos contain only pixel frames and do not provide motion vectors or depth maps. Therefore, our system cannot directly use the same information used by DLSS.

The central idea of this project is:

Can we build a DLSS-inspired video enhancement pipeline using only video frames, without explicit motion vectors?

Instead of using externally provided motion vectors, our model learns motion from pairs of consecutive frames. This makes the system more general because it can be applied to ordinary videos.

1.3 Problem Statement

Given a low-resolution and low-frame-rate input video:

$$V_{in} = \{F_1, F_2, \dots, F_N\}$$

we aim to generate an enhanced output video:

$$V_{out} = \{F'_1, F'_2, \dots, F'_M\}$$

where:

- each output frame has higher spatial resolution,
- the output video has higher temporal resolution,
- motion appears smoother,
- frame details appear sharper,
- the processing speed is suitable for real-time or near-real-time use.

This problem is divided into two major subtasks:

1. **Frame Interpolation:** Generate intermediate frames between existing frames.
2. **Super-Resolution:** Upscale each frame to a higher spatial resolution.

1.4 Core Pipeline Idea

The final pipeline can be summarized as:

$$\begin{array}{ccc} \text{Low-Resolution Low-FPS Video} & \xrightarrow{\text{Frame Interpolation}} & \text{Low-Resolution High-FPS Video} \\ & \xrightarrow{\text{Super-Resolution}} & \text{High-Resolution High-FPS Video} \end{array}$$

A simplified mathematical form is:

$$V_{out} = S_{\theta_s}(T_{\theta_t}(V_{in}))$$

where:

- T_{θ_t} is the frame interpolation model,
- S_{θ_s} is the super-resolution model,
- θ_t and θ_s are trainable model parameters.

1.5 Project Objectives

The objectives of the project are:

- To design and train a frame interpolation model capable of generating intermediate frames.
- To design and train a super-resolution model capable of $4\times$ upscaling.
- To combine both models into a single video processing pipeline.
- To use ONNX Runtime for deployment-oriented inference.
- To support hardware detection and provider selection.
- To include performance estimation before processing full videos.
- To evaluate the models using multiple quantitative metrics.
- To visually compare input, predicted, and ground truth results.
- To analyze the limitations of a frame-only DLSS-like approach.

1.6 Report Organization

Chapter 2 presents the fundamentals of digital video processing. Chapter 3 explains the mathematical formulation of the video enhancement problem. Chapter 4 discusses the theory of frame interpolation. Chapter 5 presents the theory of super-resolution. Chapter 6 explains the frame interpolation model. Chapter 7 describes the ESPCN upscaling model. Chapter 8 presents the dataset and preprocessing methods. Chapter 9 explains the training methodology. Chapter 10 discusses ONNX export and deployment. Chapter 11 describes the complete pipeline architecture. Chapter 12 discusses performance estimation and optimization. Chapter 13 presents the evaluation metrics. Chapter 14 presents the experimental results. Chapter 15 provides visual and metric-based analysis. Chapter 16 describes the development timeline. Chapter 17 discusses limitations. Chapter 18 presents future work. Chapter 19 concludes the report.

Chapter 2

Fundamentals of Digital Video Processing

2.1 Video as a Spatio-Temporal Signal

A video can be modeled as a discrete spatio-temporal signal:

$$V(x, y, t)$$

where x and y represent spatial coordinates and t represents time. For an RGB video:

$$V(x, y, t) \in \mathbb{R}^3$$

A digital video is sampled in both space and time. Spatial sampling determines image resolution, while temporal sampling determines frame rate.

2.2 Frame Representation

A video is a sequence of frames:

$$V = \{F_1, F_2, F_3, \dots, F_N\}$$

Each frame is an image:

$$F_i \in \mathbb{R}^{H \times W \times C}$$

For RGB images:

$$C = 3$$

where H is height and W is width.

2.3 Spatial Resolution

Spatial resolution refers to the number of pixels in a frame. Higher spatial resolution allows the frame to represent finer image details. If the input frame has size:

$$H \times W$$

then a $4\times$ upscaling model produces:

$$4H \times 4W$$

This increases the number of pixels by:

$$4^2 = 16$$

times.

2.4 Temporal Resolution

Temporal resolution refers to the number of frames per second. If a video is captured at f frames per second, consecutive frames are separated by:

$$\Delta t = \frac{1}{f}$$

Frame interpolation attempts to estimate missing frames at intermediate time points such as:

$$t + \frac{\Delta t}{2}$$

2.5 Why Video Enhancement is Ill-Posed

Both super-resolution and frame interpolation are ill-posed problems. In super-resolution, many high-resolution images can downsample to the same low-resolution image. In frame interpolation, many possible motion paths can exist between two observed frames.

Thus, the model must infer missing information using learned priors from data.

2.6 Spatial and Temporal Reconstruction

The full enhancement task can be written as:

$$V_{HR,HFR} = \mathcal{E}(V_{LR,LFR})$$

where:

- LR means low resolution,
- LFR means low frame rate,
- HR means high resolution,
- HFR means high frame rate,
- $\mathcal{E}$ is the enhancement pipeline.

Chapter 3

Mathematical Formulation of the Enhancement Problem

3.1 Spatial Degradation Model

A low-resolution image can be modeled as a degraded version of a high-resolution image:

$$I_{LR} = D(I_{HR}) + \eta$$

where:

- D is a degradation operator,
- η is noise,
- I_{HR} is the true high-resolution image,
- I_{LR} is the observed low-resolution image.

The degradation operator may include blur, downsampling, compression, and sensor limitations.

3.2 Temporal Degradation Model

A low-frame-rate video is a temporally sampled version of a continuous-time video signal. If the original continuous video is $V(t)$, then a low-FPS version observes only sparse samples:

$$V_{LFR} = \{V(t_0), V(t_1), V(t_2), \dots\}$$

Frame interpolation attempts to estimate:

$$V\left(\frac{t_i + t_{i+1}}{2}\right)$$

from the two neighboring frames.

3.3 Joint Enhancement Model

The complete enhancement model combines temporal and spatial reconstruction:

$$V_{out} = S(T(V_{in}))$$

where T increases the frame rate and S increases spatial resolution.

3.4 Ordering of Operations

There are two possible orders:

Upscale first $\rightarrow$ Interpolate later

or

Interpolate first $\rightarrow$ Upscale later

In this project, interpolation is applied in the low-resolution domain and upscaling is applied afterward. This has two main advantages:

- Interpolation at low resolution requires less computation.
- Upscaling after interpolation ensures all original and generated frames are enhanced consistently.

3.5 Error Propagation

If the interpolation model introduces error ϵ_T and the super-resolution model introduces error ϵ_S , then the total output error can be approximated as:

$$\epsilon_{total} \approx \epsilon_T + \epsilon_S$$

This means that poor interpolation can affect the final upscaled output. Therefore, both models must be evaluated separately and together.

Chapter 4

Theory of Frame Interpolation

4.1 Definition

Frame interpolation is the task of generating one or more intermediate frames between two existing frames. Given:

$$I_1 = I(t)$$

and

$$I_3 = I(t + \Delta t)$$

the goal is to predict:

$$I_2 = I\left(t + \frac{\Delta t}{2}\right)$$

4.2 Simple Averaging and Its Failure

A naive approach is to average the two frames:

$$\hat{I}_2 = \frac{I_1 + I_3}{2}$$

However, this fails when objects move. Moving objects appear duplicated or transparent, causing ghosting artifacts. Therefore, a successful interpolation model must estimate motion.

4.3 Optical Flow

Optical flow estimates pixel displacement between frames. If a point moves from (x, y) to $(x + u, y + v)$, then the flow vector is:

$$\mathbf{v}(x, y) = (u(x, y), v(x, y))$$

The brightness constancy assumption states:

$$I(x, y, t) = I(x + u, y + v, t + \Delta t)$$

Using first-order Taylor expansion:

$$I_x u + I_y v + I_t = 0$$

where I_x , I_y , and I_t are image derivatives.

4.4 Aperture Problem

The optical flow equation has two unknowns u and v , but only one equation. This is called the aperture problem. Additional assumptions such as smoothness are required.

Deep learning models avoid explicitly solving this equation by learning motion from data.

4.5 Occlusion

Occlusion occurs when a region visible in one frame is hidden in another. This makes interpolation difficult because the missing content may not be present in both input frames.

A learned mask helps the model decide whether to trust the first frame or the second frame more in different regions.

4.6 Warping

A frame can be warped using a flow field:

$$W(I, \mathbf{v})(x, y) = I(x + u(x, y), y + v(x, y))$$

Warping aligns pixels from input frames toward the target intermediate time.

4.7 Mask-Based Blending

After warping both frames, the model blends them:

$$I_{base} = M \cdot W(I_1, \mathbf{v}_1) + (1 - M) \cdot W(I_3, \mathbf{v}_3)$$

where M is a learned blending mask.

4.8 Residual Refinement

The base prediction may contain blur, artifacts, or incorrect boundaries. A refinement network predicts a correction:

$$\hat{I}_2 = I_{base} + R$$

The final output is clamped to the valid image range:

$$\hat{I}_2 = \text{clamp}(I_{base} + R, 0, 1)$$

Chapter 5

Theory of Super-Resolution

5.1 Definition

Single image super-resolution aims to reconstruct a high-resolution image from a low-resolution input:

$$I_{HR} = S(I_{LR})$$

where S is the learned super-resolution function.

5.2 Ill-Posed Nature

Super-resolution is ill-posed because high-frequency details are lost during downsampling. Multiple high-resolution images can produce the same low-resolution image.

5.3 Frequency Domain View

Downsampling acts like a low-pass filtering operation. High-frequency components such as edges and textures are removed. In frequency domain:

$$\mathcal{F}(I_{LR}) = \mathcal{F}(I_{HR})H(\omega)$$

where $H(\omega)$ suppresses high-frequency components.

A super-resolution model attempts to reconstruct these missing high-frequency details.

5.4 Traditional Upscaling

Traditional methods include nearest-neighbor, bilinear, and bicubic interpolation. These methods are deterministic and do not learn from data. They can enlarge the image but cannot recover realistic missing details.

5.5 Deep Learning Based Super-Resolution

Deep learning based super-resolution learns a mapping:

$$S_{\theta}(I_{LR}) = \hat{I}_{HR}$$

The training objective is:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(S_{\theta}(I_{LR}), I_{HR})$$

where $\mathcal{L}$ measures reconstruction error.

Chapter 6

Frame Interpolation Model

6.1 Input and Output

The frame interpolation model receives two RGB frames:

$$I_1, I_3 \in \mathbb{R}^{3 \times H \times W}$$

The frames are concatenated channel-wise:

$$X = [I_1, I_3] \in \mathbb{R}^{6 \times H \times W}$$

The model predicts:

$$\hat{I}_2 \in \mathbb{R}^{3 \times H \times W}$$

6.2 Encoder

The encoder extracts motion-aware features from the concatenated frame pair. Strided convolutions reduce spatial resolution and increase feature depth.

The purpose of the encoder is to learn:

- object positions,
- motion patterns,
- local texture information,
- scene structure,
- temporal differences.

6.3 Residual Blocks

Residual blocks improve learning by allowing the network to learn corrections:

$$Y = X + F(X)$$

This helps retain useful information while refining features.

6.4 Decoder

The decoder upsamples features back to the original resolution. It predicts:

- flow for the first frame,
- flow for the third frame,
- blending mask.

The output has five channels:

$$O \in \mathbb{R}^{5 \times H \times W}$$

6.5 Flow and Mask Separation

The output is split as:

$$F_1 = O_{1:2}$$

$$F_3 = O_{3:4}$$

$$M = \sigma(O_5)$$

where F_1 and F_3 are flow fields and M is the mask.

6.6 Base Frame Prediction

The warped frames are blended:

$$I_{base} = MW(I_1, F_1) + (1 - M)W(I_3, F_3)$$

6.7 Refinement Network

The refinement network receives:

$$[I_1, I_{base}, I_3]$$

This 9-channel input allows the model to compare the base prediction with both input frames. It predicts a residual correction R :

$$\hat{I}_2 = I_{base} + R$$

6.8 Why This Architecture Works

This architecture works because it combines motion alignment and learned synthesis. Warping handles geometric motion, mask blending handles visibility and occlusion, and residual refinement corrects local artifacts.

Chapter 7

Super-Resolution Model: ESPCN

7.1 Why ESPCN

ESPCN stands for Efficient Sub-Pixel Convolutional Neural Network. It is designed for real-time super-resolution.

Traditional super-resolution networks often upscale the image first and then process it. This is computationally expensive. ESPCN processes features in low-resolution space and performs upscaling only at the final stage.

7.2 Architecture

The ESPCN model contains:

- convolution layer from 3 channels to 64 feature maps,
- ReLU activation,
- convolution layer from 64 to 32 feature maps,
- ReLU activation,
- convolution layer from 32 to $3r^2$ channels,
- PixelShuffle layer.

For $4\times$ upscaling:

$$r = 4$$

The final convolution produces:

$$3 \times 4^2 = 48$$

channels.

7.3 PixelShuffle

PixelShuffle rearranges channels into spatial resolution:

$$\mathbb{R}^{Cr^2 \times H \times W} \rightarrow \mathbb{R}^{C \times rH \times rW}$$

For RGB and $4\times$ scaling:

$$48 \times H \times W \rightarrow 3 \times 4H \times 4W$$

7.4 Advantages

ESPCN was selected because:

- it is lightweight,
- it is fast,
- it has low memory usage,
- it is suitable for real-time inference,
- it can be exported to ONNX,
- it gives good PSNR and SSIM.

Chapter 8

Dataset and Preprocessing

8.1 Frame Interpolation Dataset

The frame interpolation model uses Vimeo-90K triplet data. Each sample contains:

- `im1.png`: first frame,
- `im2.png`: ground truth middle frame,
- `im3.png`: third frame.

The model input is:

$$(I_1, I_3)$$

and the target is:

$$I_2$$

8.2 Super-Resolution Dataset

The upscaling model uses paired low-resolution and high-resolution frames. The dataset contains:

- low-resolution image,
- high-resolution target image.

The model learns:

$$I_{LR} \rightarrow I_{HR}$$

8.3 Random Cropping

Random cropping is used during training to reduce memory usage and increase data variation.

For frame interpolation, the same crop is applied to all three frames to preserve alignment.

8.4 Flipping

Horizontal and vertical flips are used as augmentation. For paired data, the same transformation is applied to all corresponding images.

8.5 Normalization

Images are normalized to:

$$[0, 1]$$

This stabilizes training and inference.

Chapter 9

Training Methodology

9.1 Frame Interpolation Training

The frame interpolation model is trained to minimize the difference between the predicted intermediate frame and the ground truth middle frame.

$$\theta_t^* = \arg \min_{\theta_t} \mathcal{L}(\hat{I}_2, I_2)$$

9.2 Charbonnier Loss

Charbonnier loss is:

$$\mathcal{L}_{char} = \frac{1}{N} \sum_{i=1}^N \sqrt{(\hat{I}_2^{(i)} - I_2^{(i)})^2 + \epsilon^2}$$

It behaves like L1 loss but is smoother.

9.3 Laplacian Pyramid Loss

Laplacian loss focuses on high-frequency details:

$$\mathcal{L}_{lap} = \|L(\hat{I}_2) - L(I_2)\|_1$$

where $L(I)$ extracts edge-like high-frequency components.

9.4 Combined Loss

The final interpolation loss is:

$$\mathcal{L}_{interp} = \mathcal{L}_{char} + 0.5\mathcal{L}_{lap}$$

9.5 Super-Resolution Training

The ESPCN model is trained with L1 loss:

$$\mathcal{L}_{SR} = \frac{1}{N} \sum_{i=1}^N |S(I_{LR}^{(i)}) - I_{HR}^{(i)}|$$

L1 loss is chosen because it generally preserves sharper results than MSE.

9.6 Optimizers

The frame interpolation model uses AdamW, while the ESPCN model uses Adam. AdamW helps with regularization, while Adam provides stable convergence for the lightweight ESPCN model.

Chapter 10

ONNX Export and Deployment

10.1 Need for ONNX

PyTorch is useful for training but deployment requires efficient inference. ONNX provides a standard format for running models across different hardware backends.

10.2 Dynamic Axes

Both models are exported with dynamic input dimensions. This allows the models to accept different resolutions and batch sizes.

10.3 ONNX Runtime Providers

ONNX Runtime supports multiple execution providers:

- `CPUExecutionProvider`,
- `CUDAExecutionProvider`,
- `DirectMLExecutionProvider`,
- `MIGraphXExecutionProvider`,
- `CoreMLExecutionProvider`.

The pipeline detects available hardware and chooses a suitable provider.

10.4 CPU Fallback

GPU execution can sometimes fail due to provider incompatibility or memory issues. Therefore, the pipeline includes CPU fallback for stability.

Chapter 11

Complete Pipeline Architecture

11.1 Pipeline Overview

The complete pipeline consists of:

1. hardware detection,
2. ONNX model loading,
3. optional performance estimation,
4. video reading,
5. frame pair formation,
6. normalization,
7. padding,
8. frame interpolation,
9. upscaling,
10. tensor-to-image conversion,
11. video writing,
12. audio preservation.

11.2 Hardware Detection

The hardware detector checks the operating system and available ONNX Runtime providers. Depending on the platform, it selects CUDA, DirectML, CoreML, MIGraphX, or CPU.

11.3 Model Loading

The ONNX loader loads two models:

- frame interpolator,
- ESPCN upscaler.

Each model is loaded into an ONNX Runtime inference session.

11.4 Video Reading

The input video is opened using OpenCV. Metadata such as width, height, FPS, and frame count are extracted.

11.5 Frame Pair Processing

The video is processed as pairs of consecutive frames:

$$(F_i, F_{i+1})$$

For each pair, the system produces:

- upscaled first frame,
- upscaled interpolated frame,
- upscaled final frame if it is the last pair.

11.6 Normalization and Tensor Conversion

Frames are converted from OpenCV image format to normalized tensors:

$$[0, 255] \rightarrow [0, 1]$$

and:

$$HWC \rightarrow NCHW$$

11.7 Padding

Reflect padding is used to make dimensions compatible with the models. After inference, padding is removed.

11.8 Interpolation Stage

The interpolator receives two low-resolution frames and generates an intermediate frame.

$$\hat{F}_{i+0.5} = T(F_i, F_{i+1})$$

11.9 Upscaling Stage

The upscaler processes both original and interpolated frames:

$$F_i^{HR} = S(F_i)$$

$$\hat{F}_{i+0.5}^{HR} = S(\hat{F}_{i+0.5})$$

11.10 Video Reconstruction

The enhanced frames are converted back to RGB/BGR image format and written to an output video file with the target FPS.

11.11 Audio Preservation

Since OpenCV video writing does not preserve audio, FFmpeg is used to copy audio from the original video into the final output.

Chapter 12

Performance Estimation and Optimization

12.1 Why Performance Estimation is Needed

Before processing the entire video, the system estimates whether real-time preview is possible. It processes a small number of frame pairs and measures average processing time.

12.2 Latency

Latency is the time needed to process one frame or frame pair:

$$T_{latency}$$

Low latency is essential for preview and real-time usage.

12.3 Throughput

Throughput is:

$$FPS = \frac{1000}{T_{ms}}$$

where T_{ms} is the processing time in milliseconds.

12.4 Preview Mode

If estimated FPS is greater than the target FPS, preview mode is possible. Otherwise, export mode is preferred.

12.5 Caching

Caching avoids recomputing previously processed frames. It is especially useful in preview systems where users may replay or seek through videos.

12.6 Batch Size Detection

The pipeline can test increasing batch sizes to find a stable value that fits memory constraints.

Chapter 13

Evaluation Metrics

13.1 Need for Multiple Metrics

Video enhancement quality cannot be judged using a single metric. PSNR measures pixel accuracy, but it does not fully reflect perceptual quality. SSIM measures structure, LPIPS measures perceptual similarity, and latency measures real-time capability.

13.2 MSE

$$MSE = \frac{1}{N} \sum_{i=1}^N (I_i - \hat{I}_i)^2$$

13.3 MAE

$$MAE = \frac{1}{N} \sum_{i=1}^N |I_i - \hat{I}_i|$$

13.4 RMSE

$$RMSE = \sqrt{MSE}$$

13.5 PSNR

$$PSNR = 20 \log_{10} \left(\frac{MAX}{\sqrt{MSE}} \right)$$

For normalized images, $MAX = 1$.

13.6 SSIM

SSIM compares luminance, contrast, and structure:

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

13.7 MS-SSIM

MS-SSIM extends SSIM across multiple scales.

13.8 LPIPS

LPIPS compares deep feature representations. Lower LPIPS indicates better perceptual similarity.

13.9 Latency and FPS

Latency and FPS measure real-time performance.

13.10 Model Size and Parameter Count

These indicate deployment efficiency.

Chapter 14

Experimental Results

14.1 Upscaling Model Results

The ESPCN upscaling model was evaluated on 7824 image pairs.

Metric	Value
Total Test Images	7824
Average PSNR	31.02 dB
Average SSIM	0.9847
Average MSE	0.001286
Average MAE	0.018320
Average RMSE	0.032120
Inference Time	0.97 ms
Estimated FPS	1031.59 FPS
Trainable Parameters	37,200
Model Size	0.14 MB

Table 14.1: ESPCN super-resolution evaluation results.

14.2 Interpretation of Upscaling Results

The PSNR value of 31.02 dB indicates good pixel-level reconstruction. The SSIM value of 0.9847 indicates excellent structural similarity. The inference time of 0.97 ms shows that the ESPCN model is extremely fast and suitable for real-time applications.

14.3 Frame Interpolation Results

The frame interpolation model was evaluated on 3783 test samples.

Metric	Value
PSNR	33.8689 dB
SSIM	0.9521
MS-SSIM	0.9832
LPIPS	0.0495
MSE	0.000769
RMSE	0.0240
Average Latency	14.68 ms
Maximum Throughput	68.1 FPS
PSNR Standard Deviation	5.2640
Peak VRAM Usage	260.30 MB

Table 14.2: Frame interpolation evaluation results.

14.4 Interpretation of Interpolation Results

The PSNR value of 33.8689 dB shows that the predicted intermediate frames are close to ground truth frames. SSIM and MS-SSIM indicate that the model preserves structural details well. LPIPS of 0.0495 indicates strong perceptual similarity. The throughput of 68.1 FPS demonstrates that the model can support real-time or near-real-time usage.

14.5 Combined Pipeline Interpretation

The combined system improves both frame rate and resolution. The interpolation model increases temporal smoothness, while the upscaling model improves spatial sharpness. Since the upscaler is faster than the interpolator, the interpolation stage is the main performance bottleneck.

Chapter 15

Visual Analysis

15.1 Importance of Visual Results

Numerical metrics do not fully represent perceived video quality. Therefore, the report should include visual comparisons.

15.2 Frame Interpolation Comparison

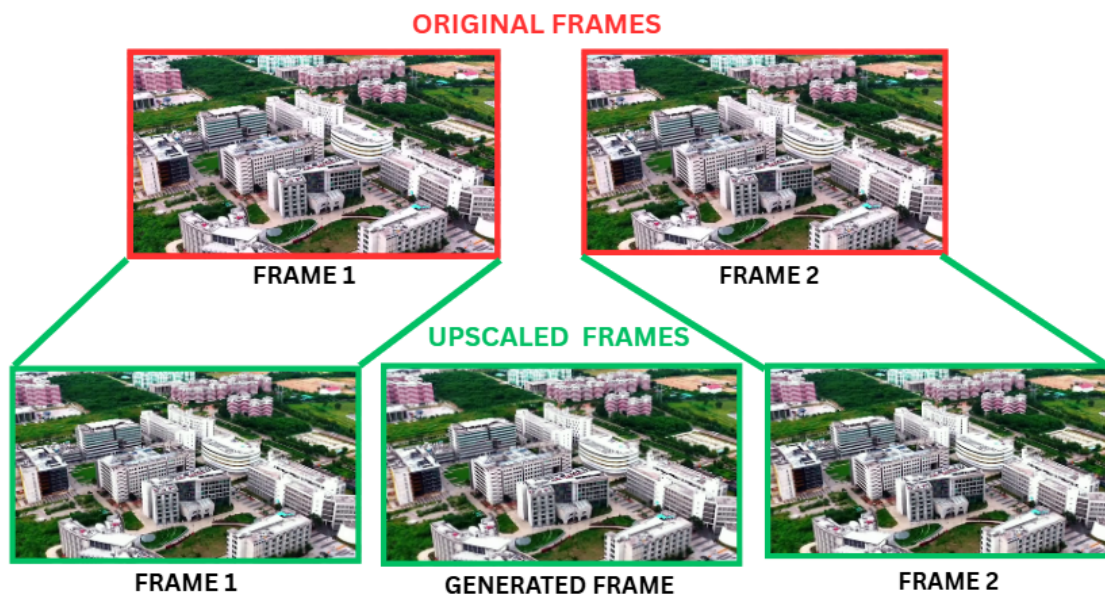


Figure 15.1: Frame interpolation visual comparison.

15.3 Super-Resolution Comparison

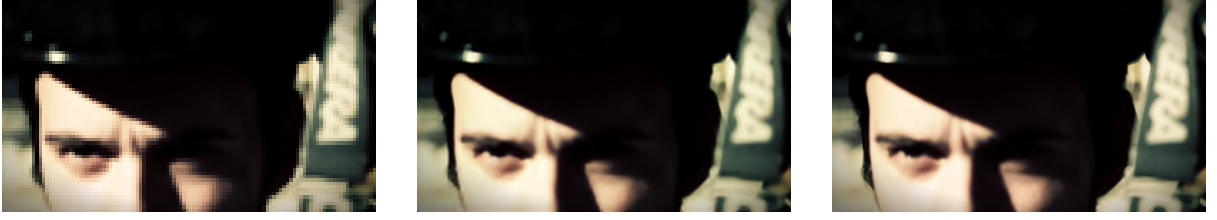


Figure 15.2: Super-resolution comparison: low-resolution input, ESPCN output, and ground truth.

15.4 Zoomed Crop Analysis

Zoomed crops are important for showing edge reconstruction.



Figure 15.3: Zoomed comparison showing improvement in local details.

15.5 Metric Graphs

Quantitative evaluation was performed for both major components of the proposed video enhancement pipeline: the super-resolution module and the frame interpolation module. The super-resolution module was evaluated using PSNR, SSIM, and inference latency, while the frame interpolation module was evaluated using PSNR, SSIM, MS-SSIM, LPIPS, interpolation error, and latency-based performance metrics.

The objective of this analysis is to study both the visual reconstruction quality and the

practical computational efficiency of the complete system. PSNR and SSIM evaluate image reconstruction accuracy and structural preservation, LPIPS measures perceptual similarity, and latency/FPS indicate whether the system is suitable for real-time or near-real-time video enhancement.

15.5.1 Super-Resolution Metric Analysis

The super-resolution model is responsible for improving the spatial resolution of individual frames. In this project, the ESPCN model was used for image upscaling because of its lightweight architecture and fast inference performance.

PSNR Distribution for Super-Resolution

Peak Signal-to-Noise Ratio measures the pixel-level reconstruction quality between the generated super-resolved image and the corresponding high-resolution target image. A higher PSNR value generally indicates lower reconstruction error and better restoration quality.

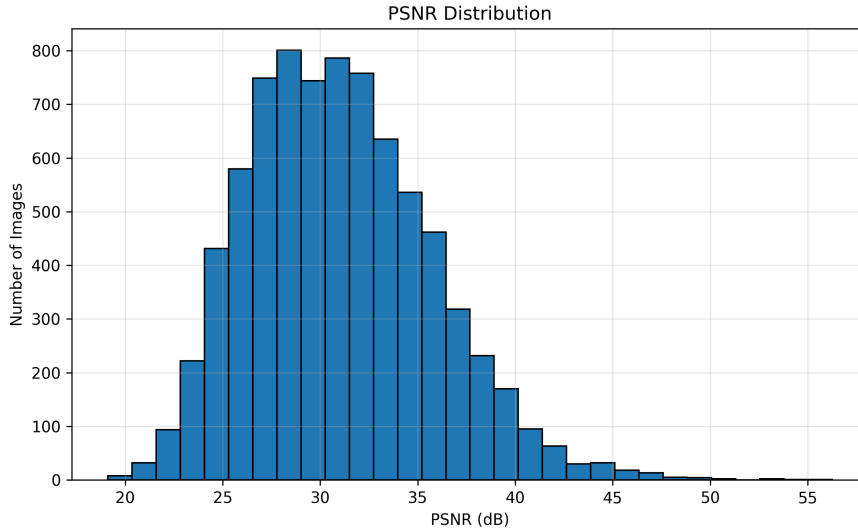


Figure 15.4: PSNR distribution for the ESPCN super-resolution model across the test samples. Most samples lie in a good PSNR range, indicating that the model produces reconstructions with relatively low pixel-wise distortion.

From Figure 15.4, it can be observed that most test samples achieve PSNR values approximately in the range of 28–35 dB. This shows that the ESPCN model is able to reconstruct the high-resolution target images with good pixel-level accuracy. A small number of samples achieve very high PSNR values, which may correspond to visually simpler frames with fewer high-frequency textures or less complex image content.

SSIM Distribution for Super-Resolution

Structural Similarity Index Measure evaluates the perceptual similarity between two images by comparing luminance, contrast, and structural information. Unlike PSNR, which mainly focuses on pixel-wise error, SSIM gives a better indication of how visually similar

the reconstructed image is to the ground-truth image.

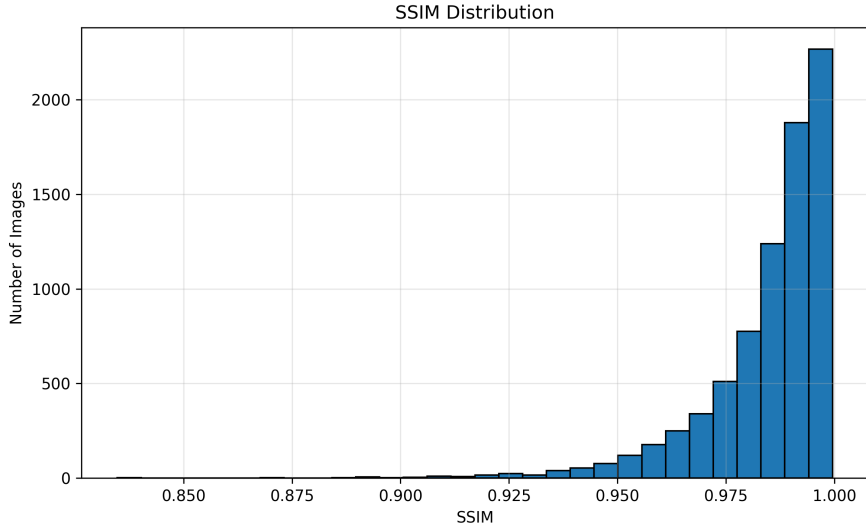


Figure 15.5: SSIM distribution for the ESPCN super-resolution model. The values are highly concentrated close to 1.0, showing strong structural similarity between the generated outputs and the ground-truth images.

As shown in Figure 15.5, the SSIM values are concentrated close to 1.0. This indicates that the super-resolution model preserves important structural details such as edges, object boundaries, and spatial patterns effectively. The strong SSIM distribution suggests that the model performs well not only in terms of numerical reconstruction but also in terms of perceived visual structure.

Inference Latency for Super-Resolution

Latency measures the time required by the model to process a single input frame. For real-time video enhancement, lower latency is essential because each frame must be processed within a very small time window.

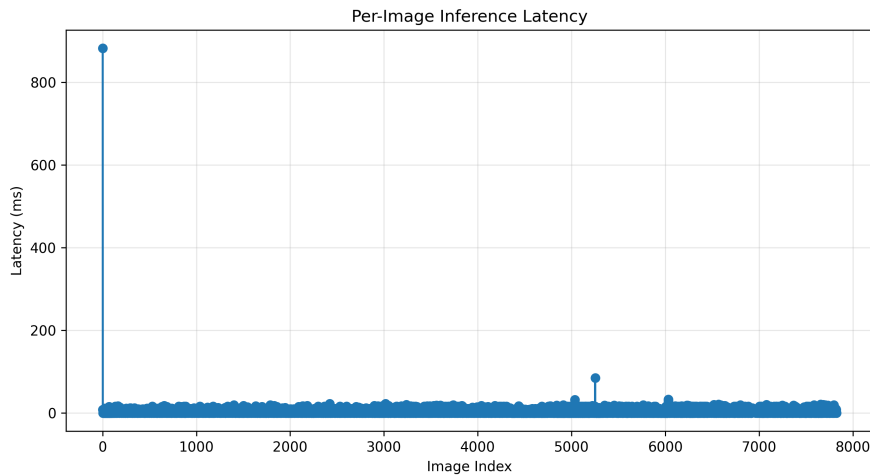


Figure 15.6: Per-image inference latency of the ESPCN super-resolution model. The initial spike is mainly due to GPU warm-up and CUDA initialization, while the remaining frames show stable inference time.

From Figure 15.6, an initial latency spike can be observed at the beginning of inference. This is expected during GPU-based execution because the first few frames may include CUDA initialization, memory allocation, and model warm-up overhead. After this initial stage, the latency becomes stable and remains low for most test samples. This indicates that the ESPCN model is computationally efficient and suitable for real-time or near-real-time super-resolution.

15.5.2 Frame Interpolation Metric Analysis

The frame interpolation model is responsible for improving the temporal resolution of the input video by generating intermediate frames between consecutive frames. This component directly affects motion smoothness and temporal continuity in the final enhanced output video.

PSNR Stability Across Interpolation Samples

The PSNR stability plot shows how the interpolation model performs across the complete test sequence. Instead of reporting only a single average value, this plot helps identify whether the model performs consistently or whether there are large variations across different samples.

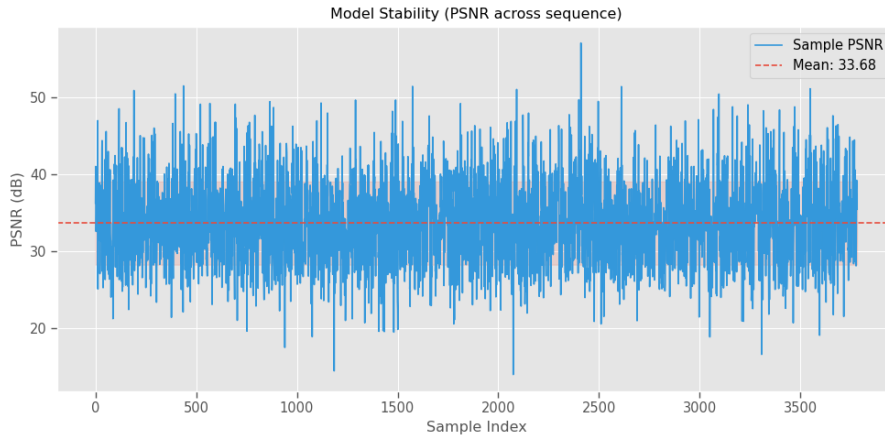


Figure 15.7: PSNR stability analysis for the frame interpolation model across the test sequence. The dashed horizontal line represents the mean PSNR, while variations around it show sample-wise performance changes.

Figure 15.7 shows that the frame interpolation model maintains a strong average PSNR of approximately 33.8689 dB across the test samples. Although there are fluctuations between frames, most predictions remain within a high-quality reconstruction range. The variation is expected because some video samples contain simple motion, while others contain complex motion, occlusion, blur, or high-frequency textures.

PSNR and LPIPS Correlation

PSNR measures pixel-level accuracy, while LPIPS measures perceptual similarity using deep feature representations. A good interpolation model should ideally produce high PSNR and low LPIPS values.

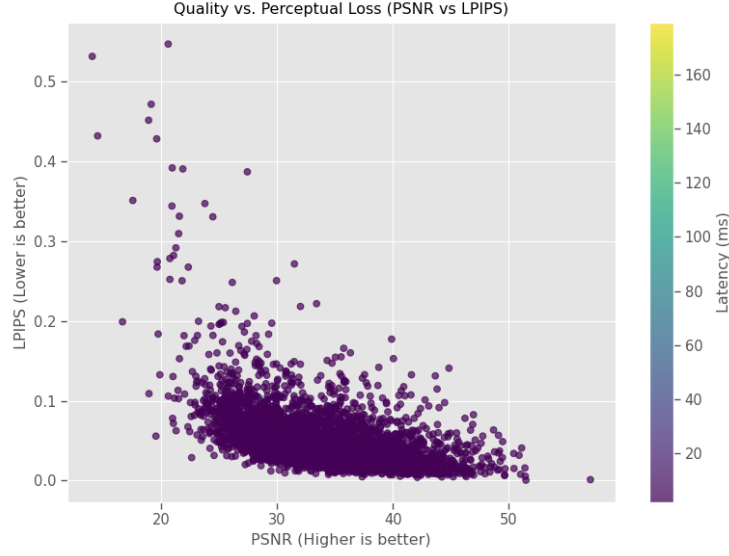


Figure 15.8: Quality and perceptual loss comparison for frame interpolation. Higher PSNR and lower LPIPS indicate better reconstruction and stronger perceptual similarity.

Figure 15.8 shows that samples with higher PSNR generally correspond to lower LPIPS values. This indicates that frames with better pixel-level reconstruction also tend to be perceptually closer to the ground-truth frames. The clustering of most samples at low LPIPS values shows that the interpolation model produces visually reliable intermediate frames for a large portion of the dataset.

Steady-State Latency Distribution

For real-time video enhancement, latency is one of the most important performance metrics. The first few frames may include hardware warm-up overhead, so steady-state latency gives a more realistic estimate of actual runtime performance.

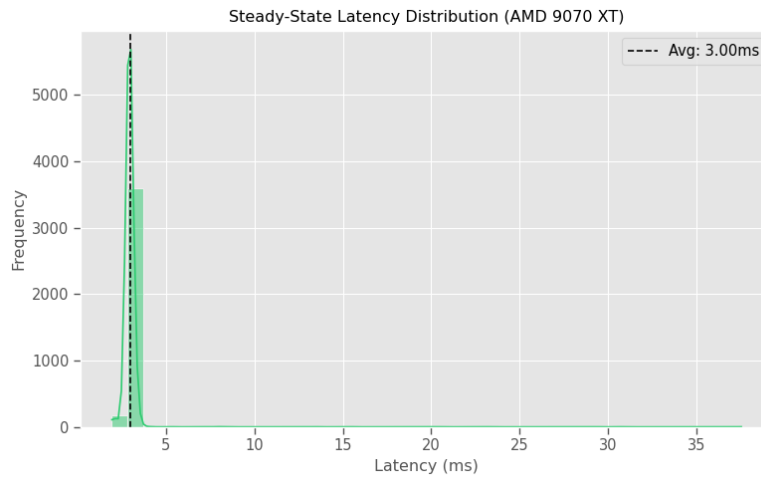


Figure 15.9: Steady-state latency distribution for the frame interpolation model. The plot excludes the initial warm-up overhead and represents the stabilized inference behavior after the model has already started running.

Figure 15.9 shows the steady-state latency behavior of the interpolation model after

warm-up. This indicates that after the initial warm-up stage, the model can process frames efficiently. Low and stable latency is important because the interpolation model is usually the main computational bottleneck in a video enhancement pipeline.

Cumulative Distribution of PSNR

The cumulative distribution function of PSNR shows what fraction of frames achieve a PSNR below or above a given threshold. This provides a clearer understanding of how consistently the model performs across the dataset.

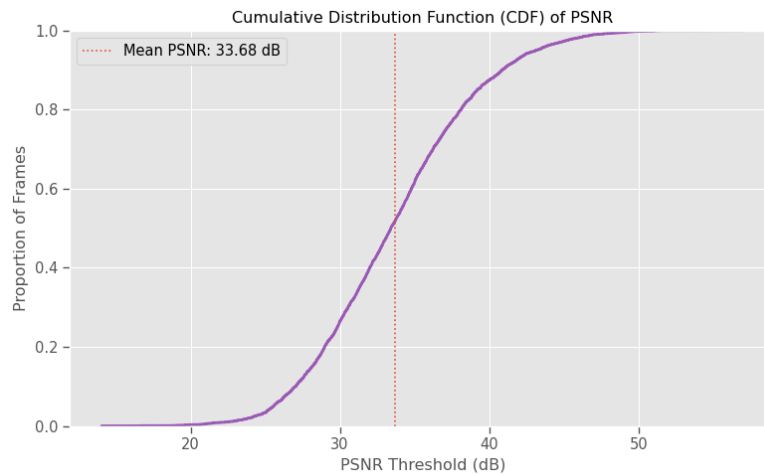


Figure 15.10: Cumulative distribution function of PSNR for the frame interpolation model. The curve shows the proportion of frames below each PSNR threshold.

From Figure 15.10, it can be observed that a large portion of frames lie around or above the mean PSNR value. The mean PSNR is approximately 33.8689 dB, which indicates good reconstruction quality for interpolated frames. The curve also shows that only a small fraction of frames fall in the low-PSNR region, suggesting that the model performs reliably across most test samples.

Correlation Heatmap of Interpolation Metrics

A correlation heatmap helps understand how different evaluation metrics are related. This is useful because different metrics capture different aspects of frame quality.

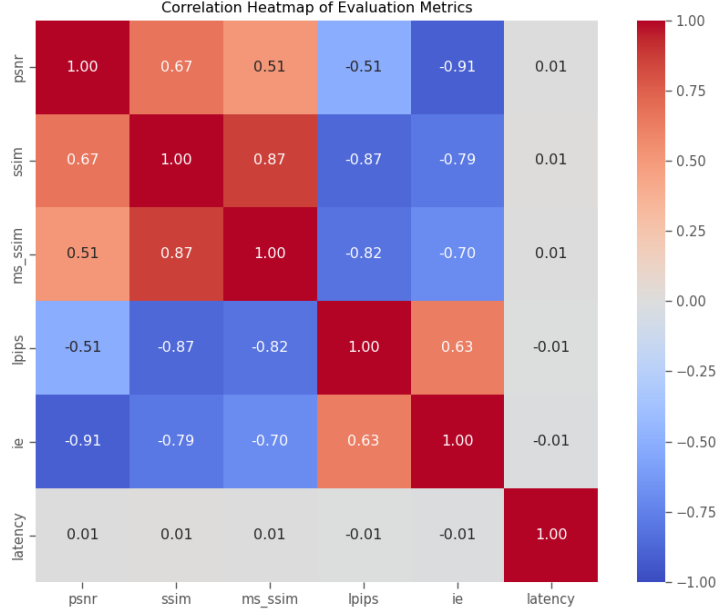


Figure 15.11: Correlation heatmap of frame interpolation metrics. PSNR, SSIM, MS-SSIM, LPIPS, interpolation error, and latency are compared to understand metric relationships.

Figure 15.11 shows that PSNR, SSIM, and MS-SSIM are positively correlated, which is expected because all three metrics improve when the reconstructed frame becomes closer to the ground truth. LPIPS shows a negative correlation with SSIM and MS-SSIM because lower LPIPS indicates better perceptual similarity. Interpolation error is strongly negatively correlated with PSNR, showing that lower error leads to better reconstruction quality. Latency has almost no correlation with the quality metrics, which suggests that frame quality is not strongly dependent on inference time for individual samples.

15.5.3 Combined Metric Summary

The super-resolution and frame interpolation results together show that the proposed pipeline improves both spatial and temporal quality. The ESPCN model provides fast and effective spatial upscaling, while the interpolation model generates visually consistent intermediate frames with strong PSNR, SSIM, and LPIPS behavior.

Table 15.1: Summary of major quantitative results for the complete video enhancement pipeline.

Metric	Super-Resolution	Frame tion	Interpola- tion
Main Purpose	Spatial enhancement	Temporal ment	enhance- ment
Average PSNR	31.02 dB	33.8689 dB	
Average SSIM	0.9847	0.9521	
MS-SSIM	–	0.9832	
LPIPS	–	0.0495	
Average Latency	0.97 ms/image	14.68 ms/frame	
Estimated Throughput	1031.59 FPS	68.1 FPS	
Model Size	0.14 MB	–	
Main Observation	Fast and structurally accurate upscaling	Stable and perceptu- ally strong intermedi- ate frame generation	

The combined results indicate that the overall pipeline achieves a good balance between reconstruction quality and computational efficiency. The super-resolution model achieves high SSIM and very low latency, making it suitable for fast spatial enhancement. The interpolation model achieves strong PSNR and perceptual similarity while maintaining real-time capable throughput. Therefore, the proposed system is suitable for real-time or near-real-time enhancement of ordinary videos using only frame-based information.

15.5.4 Overall Interpretation

The metric graphs show that both components of the pipeline perform effectively. The PSNR and SSIM distributions for super-resolution indicate that the ESPCN model reconstructs high-resolution frames with good pixel-level accuracy and excellent structural preservation. The interpolation plots show that the frame generation model maintains stable quality across thousands of samples, produces low perceptual error, and achieves efficient inference after warm-up.

Overall, the system demonstrates that a DLSS-inspired enhancement approach can be implemented without motion vectors, depth buffers, or temporal rendering history. Instead, the system learns motion and spatial reconstruction directly from video frames. This makes the proposed method practical for ordinary videos where only frame data is available.

Chapter 16

Development Timeline

16.1 Initial Stage

The project began with basic frame interpolation and upscaling experiments.

16.2 Dataset Setup

Vimeo datasets were used for both interpolation and upscaling experiments.

16.3 Model Exploration

For upscaling, FSRCNN and ESPCN were explored. ESPCN was selected for its speed and efficiency.

For interpolation, flow-based and direct interpolation ideas were explored. The final model uses flow-like prediction, mask blending, and residual refinement.

16.4 Loss Improvements

Charbonnier and Laplacian losses were used to improve interpolation sharpness. L1 loss was used for ESPCN.

16.5 ONNX Deployment

Both models were exported to ONNX with dynamic input dimensions.

16.6 Pipeline Integration

The models were combined into a single pipeline that reads video, interpolates frames, upscales frames, and writes final output.

16.7 Final Improvements

The pipeline was improved with hardware detection, provider selection, preview support, caching design, progress reporting, and audio copy support.

Chapter 17

Limitations

17.1 Frame Interpolation Limitations

The interpolation model may struggle with:

- large motion,
- occlusion,
- transparent objects,
- fast camera movement,
- complex textures,
- motion blur.

17.2 Super-Resolution Limitations

The super-resolution model may struggle with:

- very blurry input,
- high compression artifacts,
- fine text,
- hallucination of incorrect details,
- temporal inconsistency between frames.

17.3 Pipeline Limitations

Errors from interpolation can be amplified by upscaling. Also, processing high-resolution videos requires more memory and computation.

Chapter 18

Future Work

Future improvements include:

- multi-frame interpolation,
- temporal consistency loss,
- transformer-based video models,
- recurrent video super-resolution,
- better occlusion handling,
- TensorRT optimization,
- GUI application,
- real-time preview player,
- mobile deployment,
- end-to-end joint training of interpolation and upscaling.

Chapter 19

Conclusion

This project presents a complete real-time video enhancement pipeline that combines frame interpolation and super-resolution. The system is inspired by DLSS-like enhancement, but unlike DLSS, it does not rely on motion vectors or depth buffers. Instead, it learns temporal motion directly from frame pairs and reconstructs spatial detail using ESPCN-based super-resolution.

The frame interpolation model improves temporal resolution by generating intermediate frames, while the ESPCN model improves spatial resolution through efficient $4\times$ upscaling. The complete pipeline integrates both models using ONNX Runtime and includes deployment-oriented features such as hardware detection, provider selection, performance estimation, caching logic, preview support, and audio preservation.

The results show that both models achieve good quality and practical speed. The interpolation model achieves strong PSNR, SSIM, MS-SSIM, and LPIPS results while maintaining real-time capable throughput. The upscaling model achieves high SSIM and very high inference speed with a small model size.

Overall, the project demonstrates that a DLSS-inspired video enhancement system can be built for ordinary videos using only frame-based deep learning methods. While limitations remain in handling complex motion and temporal consistency, the system provides a strong foundation for future work in real-time video enhancement.

Bibliography

- [1] W. Shi et al., “Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network,” IEEE Conference on Computer Vision and Pattern Recognition, 2016.
- [2] C. Dong et al., “Image Super-Resolution Using Deep Convolutional Networks,” IEEE Transactions on Pattern Analysis and Machine Intelligence, 2016.
- [3] A. Dosovitskiy et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” IEEE International Conference on Computer Vision, 2015.
- [4] T. Xue, B. Chen, J. Wu, D. Wei, and W. T. Freeman, “Video Enhancement with Task-Oriented Flow,” International Journal of Computer Vision, 2019.
- [5] R. Zhang et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” IEEE Conference on Computer Vision and Pattern Recognition, 2018.
- [6] ONNX Developers, “Open Neural Network Exchange,” <https://onnx.ai/>
- [7] PyTorch Developers, “PyTorch Documentation,” <https://pytorch.org/>